HalluciGuard

Complete Project Guide

Detecting, locating and explaining hallucinations in a Retrieval-Augmented Generation (RAG) system

Everything you need to understand the project from scratch: the ideas behind it, how every component works, what the results mean, its limitations, and how to answer questions about it. No coding knowledge is required.

Updated edition, matching the code at commit b7724c4 (2026-10-11). Exact rules and numbers: FACTS.md.

How to use this guide

This guide is written for someone who has never seen the project before. It starts with the basic ideas, follows one real question through every step of the system, explains how the system is tested and what the results mean, and how to defend it.

If you want to...	Read
Get the idea in one minute	Section 1.1
Understand the concepts (embeddings, NLI, attention...)	Part A, sections 1-4
Understand how a question flows through the system	Part B, sections 5-14
Understand the testing and the numbers	Part C, sections 15-18
Prepare for questions from a professor	Part D, sections 19-23
Run the demo or find a file in the code	Part E, sections 24-26
See what changed from the earlier edition	The next page

Coloured boxes highlight the most important points:

Key idea

The core concepts you must understand.

What to say

Ready-made sentences for explaining or answering.

Be careful

Common mistakes, wrong wording, or things that could get you into trouble.

Example

Worked examples with concrete numbers.

Check yourself

A short question to test your understanding; the answer follows.

About the numbers

Numbers come from runs on 2026-10-11 and are dated where they appear. Some figures are marked **illustrative**: made-up but realistic numbers that explain an idea. Before presenting, check the tables in section 18 against README.md and FACTS.md, and read section 18.5.

What changed in this edition

The earlier edition described the project as it was part-way through the build. This edition matches the final code. The main differences:

Topic	Earlier edition said	Now (the code)
Gemini	"The system can still be switched back to Gemini with a setting"	Gemini was **removed completely**; Ollama (qwen2.5:3b) is the only generator
When a sentence is red	Red if any chunk gives contradiction above 0.5, or best entailment below 0.5	Red only if the best entailment is below 0.5. A contradiction label alone gives amber. Wrong numbers or names make a sentence amber with bright red words (section 10.8)
Number checker	Normalises "1,600" = "1.6 thousand", matches dates by year	Only commas are removed; no unit or magnitude handling. Numbers are checked against the passage the sentence is about, chosen by meaning (section 11.2)
Claims and filler	Not present	Compound sentences are split into claims; filler sentences are grey and excluded from scoring (10.6, 10.7)
Document authority	Hand-set per document (e.g. 0.9 and 0.6)	All six documents are 0.8, so the document score is uninformative (6.3, 13.3)
Loose mode	Switched on in the dashboard	An environment variable (`PROMPT_MODE=loose`); only faulty mode is a dashboard checkbox (16)
Attention	For every red sentence	Only for sentences NLI rejected (12)
Threshold 0.5	"Not tuned; a sweep would tune it"	Swept from 0.3 to 0.9: F1 flat at 0.71-0.72 (18.3)
Evaluation	22 questions and the adversarial test	Plus a labelled set of 176 sentences (AI-labelled) with precision, recall, F1, confidence intervals, an ablation and AUROC for the trust parts (18.3)
Speed	About 2.5 s per answer	2.5 s generation; about 8 s for a whole query; models preloaded at startup
Code map	rag_pipeline.py runs the whole flow	rag_pipeline.py only retrieves, prunes and generates; query_service.py and verification.py do the rest (25)
Trust example	Used a Gemini answer (0.75)	Uses a faulty-mode answer (0.69); known edge cases include 0.00 for one flagged claim (13.9)
Not built	Pronoun retry and the attention masking check were listed as ideas	Still not built; stated as such (10.6, 12.2, 18.4)

Contents

Part A: Foundations: the problem and the key ideas 7

1. The problem: AI hallucinations 8
- 1.1 The one-minute summary 8
- 1.2 What exactly is a hallucination? 8
- 1.3 Why it matters 8

2. The big idea: an open-book exam with a teacher 9
- 2.1 Prevent, detect, explain 9

3. The most important idea: “supported” is not the same as “true” 10

4. Key AI concepts you need 11
- 4.1 Tokens 11
- 4.2 Embeddings: turning meaning into numbers 11
- 4.3 Probabilities, logprobs and softmax 11
- 4.4 Transformers and attention 12
- 4.5 Classifiers vs generators 12

Part B: How the system works, step by step 13

5. The system at a glance 14
- 5.1 The main parts of the software 14
- 5.2 The API endpoints 14

6. Step 0: Preparing the documents (ingestion) 16
- 6.1 The corpus 16
- 6.2 Chunking 16
- 6.3 Embedding and storing 16
- 6.4 The database tables 16

7. Step 1: Retrieval — finding the right chunks 18
- 7.1 Why embeddings instead of word search? 18
- 7.2 The weakness of retrieval 18

8. Step 2: MI pruning — removing useless chunks 19
- 8.1 The question pruning asks 19
- 8.2 PMI in plain words 19
- 8.3 The pruning rules 19
- 8.4 Why prune? 20
- 8.5 Why PMI and not a trained model? 20
- 8.6 The result 20

9. Step 3: Generation — the AI writes the answer 21
- 9.1 What the generator receives 21
- 9.2 Logprobs: the model's confidence 21
- 9.3 The cache 21
- 9.4 Why a local model (Ollama) and not an online API? 21

10. Step 4: NLI verification — checking each sentence 23
- 10.1 What NLI is 23
- 10.2 Why the chunk is the premise and the sentence is the hypothesis 23
- 10.3 The model: DeBERTa 23
- 10.4 What happens inside, step by step 23

10.5 Checking a whole answer: the sentence × chunk table 24
10.6 Atomic claims: splitting compound sentences 24
10.7 Filler sentences 24
10.8 The decision rule (exactly how a sentence gets its colour) 25
10.9 Why sentence by sentence? 25
10.10 Similar words are not the same meaning 25
10.11 Where NLI makes mistakes 25

11. Step 5: The number and name checker (span check) 27
11.1 What it checks 27
11.2 Exactly how matching works 27
11.3 Its limits, and why it is still useful 28

12. Step 6: Attention attribution — where was the checker looking? 29
12.1 How it is computed 29
12.2 The correct way to describe it 29

13. Step 7: The hierarchical trust score 30
13.1 Sentence score 30
13.2 Entity score 30
13.3 Document score 30
13.4 Token-probability score 30
13.5 A worked example 31
13.6 When a component is missing: renormalisation 31
13.7 Why “hierarchical”, and why combine four signals? 31
13.8 How the weights were chosen 32
13.9 Known edge cases 32

14. Step 8: Saving and showing the result 33
14.1 The dashboard 33

Part C: Testing the system and understanding the results 34

15. The adversarial test: does the checker actually work? 35
15.1 Normal use vs testing 35
15.2 How it works 35
15.3 The attack types 36
15.4 The control set: why it is essential 36
15.5 Why not corrupt the documents and ask normal questions? 36
15.6 Interpreting the results 36

16. Test modes: faulty mode and loose mode 38

17. How the system is measured: the metrics 39
17.1 The evaluation set 39
17.2 Metric definitions 39
17.3 Why precision@6 can never exceed 0.33 here 39

18. The results 40
18.1 Results table 40
18.2 What the results show 41
18.3 Labelled-set evaluation and ablation 41
18.4 The pronoun problem in detail 43
18.5 Things to double-check before presenting 43

Part D: Defending the project: limitations, decisions and questions 44

19. Limitations 45

20. Design decisions and the alternatives considered 46

21. Future work 47

22. Question bank: likely questions and answers 48

22.1 The project as a whole 48

22.2 Retrieval and pruning 48

22.3 Verification 49

22.4 Attention and trust 50

22.5 Testing 50

23. Wording guide: say this, not that 52

Part E: Running the demo and finding your way around the code 53

24. Running the demo 54

24.1 Before presenting 54

24.2 Demo script (about 3 minutes) 54

24.3 Troubleshooting 54

25. Where things are in the code 56

25.1 Suggested reading order for the code 57

26. Glossary 58

Appendix: the running example, end to end 60

Part A

Foundations: the problem and the key ideas

1. The problem: AI hallucinations

1.1 The one-minute summary

AI language models (like ChatGPT or the model used here) sometimes state things that are not true, and they say them just as confidently as things that are true. This is called a **hallucination**.

HalluciGuard answers questions using a small set of documents, then **checks every sentence of its own answer against those documents**. It colours each sentence (green = supported, amber = neutral or containing wrong parts, red = unsupported, grey = filler with no claim), marks the *exact wrong words* (numbers and names the sources don't contain) in bright red, shows which source words the checker focused on, and gives the whole answer a **trust score** between 0 and 1.

> **The one-paragraph explanation (memorise this)**
> "HalluciGuard answers questions from 6 renewable-energy documents. It finds the relevant chunks, removes useless ones (pruning), and a local AI model (qwen2.5:3b running on Ollama) writes an answer. Then a checker model (DeBERTa NLI) tests each sentence, or each smaller claim inside it, against the sources, a second check looks for numbers and names the sources do not contain, unsupported parts are marked, and a trust score is computed. We test the checker with sentences we broke on purpose and with a labelled set of real answers."

1.2 What exactly is a hallucination?

A language model writes text by repeatedly predicting the next word. It learned which words usually follow which from huge amounts of text. It does not look facts up in a database; it produces text that *sounds* right. Usually that text is also correct, but sometimes it is fluent and wrong. Typical hallucinations:

- **Wrong numbers or dates:** "345 GW" becomes "120 GW", or "2023" becomes "2019".
- **Invented details:** a plausible-sounding name, statistic or quotation that appears nowhere in the sources.
- **Mixed-up facts:** a fact about wind power stated as if it were about solar power.
- **Over-generalisation or commentary:** "some countries" becomes "all countries", or opinions such as "this underscores the country's commitment".

The danger is that the user cannot tell the difference from the way the answer is written. A wrong sentence looks exactly as confident as a right one.

1.3 Why it matters

If people use AI answers for studying, business, medicine or law, a confident wrong answer can cause real harm. Organisations that use AI on their own documents need to know **which parts of an answer actually come from their documents** and which parts the AI added on its own. HalluciGuard addresses exactly this need.

2. The big idea: an open-book exam with a teacher

Imagine a student taking an open-book exam:

1. The student gets a question.
2. They look up the relevant pages in the book.
3. They write an answer using those pages.
4. **A teacher then checks the answer line by line**, asking for each line: “Does the book actually say this?”

In the exam	In HalluciGuard
The book	6 documents about renewable energy (the **corpus**)
Looking up the relevant pages	**Retrieval**: finding the most relevant pieces of the documents
Throwing away pages that aren't useful	**MI pruning**: removing chunks that don't really match the question
The student writing the answer	The **generator**: qwen2.5:3b, running locally through Ollama
The teacher checking each line	The **verifier**: DeBERTa (NLI) plus a number-and-name checker
The teacher's overall grade	The **trust score**

Steps 1 to 3 are a standard technique called **RAG** (Retrieval-Augmented Generation): “retrieval” means looking things up, “augmented” means the AI gets extra information, and “generation” means writing the answer.

> **Key idea**
> Most RAG systems stop after the student writes the answer. **HalluciGuard adds the teacher.** The checking part (verification, wrong-part marking, attention, trust score) is the main contribution of the project. The RAG part only needs to work properly; it is the setting in which the checker operates.

2.1 Prevent, detect, explain

Job	How	Components
Prevent hallucinations a little	Give the AI only relevant, non-repetitive context	MI pruning
Detect hallucinations (the main job)	Check every sentence (and each claim inside it) against the sources	NLI verifier, claim splitting, number and name checker
Explain them	Show where and why, and summarise reliability	Red words, attention words, trust score, dashboard

3. The most important idea: “supported” is not the same as “true”

HalluciGuard checks whether a sentence is **supported by the documents**. It does **not** check whether the sentence is true in the real world. Professors very often ask about this.

	Question	Can we check it?
True (factual)	Is it correct in the real world?	Very hard: you would need all the world's knowledge, always up to date
Supported (grounded / faithful)	Do *our documents* back it up?	Yes: this is what HalluciGuard does

> **Example**
> Our documents say nothing about the capital of Australia. If the AI writes “Australia has Canberra as its capital”, the sentence is **true**, but the documents don't say it, so it is marked unsupported. (This exact sentence appeared in our test data.) The opposite can also happen: if a document contained a mistake and the AI copied it, the sentence would be marked **supported** even though it is false.

So in this project, red does not mean “false”. It means: **“this cannot be verified from the sources, so don't trust it blindly.”** That includes true facts from the model's own knowledge *and* unsupported commentary such as “this achievement underscores the country's commitment to renewable energy”.

> **Key idea**
> In this project: **Hallucination = a claim that is not supported by the source documents.**
> Researchers call this property *faithfulness* or *groundedness*.

> **What to say**
> Question: “Your system marked a true sentence red. Isn't that a mistake?”
> Answer: “No. We detect unsupported claims, not false ones. The sentence may be true, but the sources don't back it, so the user can't verify it.”

4. Key AI concepts you need

Each concept is explained without maths first; where a formula helps, it is explained in words.

4.1 Tokens

AI models do not read whole words; they read **tokens**, which are words or pieces of words. “solar” might be one token, while “gigawatts” might be split into “giga” + “watts”. A rough rule: 100 tokens ≈ 75 English words. Models can only read a limited number of tokens at once. The checker model (DeBERTa) reads at most **512 tokens** (the code truncates the chunk, never the sentence). This limit shapes the chunk size.

4.2 Embeddings: turning meaning into numbers

An **embedding** is a list of numbers that represents the meaning of a piece of text. HalluciGuard uses a small model called **MiniLM** (all-MiniLM-L6-v2), which turns any text into a list of **384 numbers**. Texts with similar meaning get similar numbers: picture each text as a point on a map, where solar texts cluster together, wind texts cluster elsewhere, and so on.

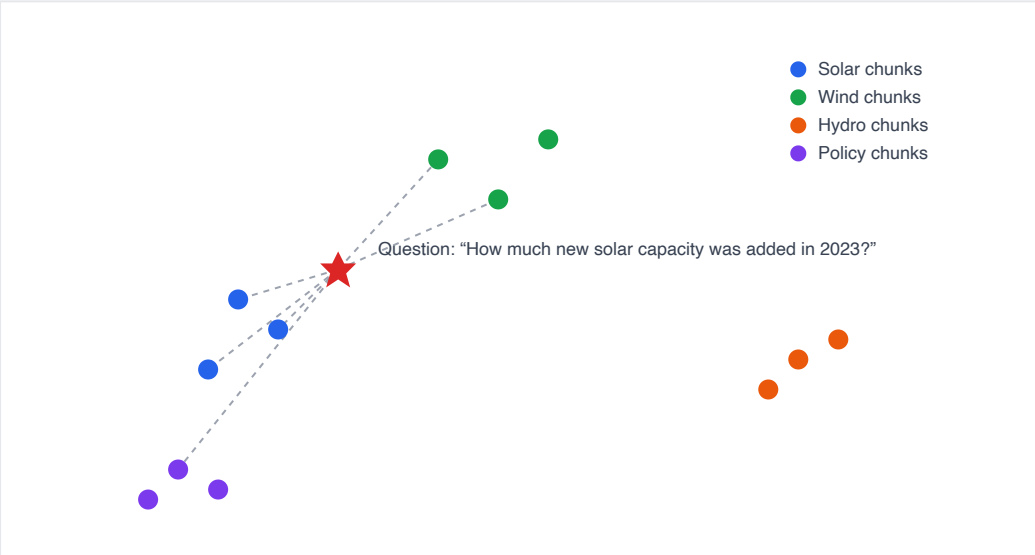


Figure 1. Illustration of embeddings as a meaning map (real embeddings have 384 dimensions, not 2). Dashed lines = the 6 nearest chunks.

Cosine similarity measures how close two embeddings are: think of each embedding as an arrow, and ask how much two arrows point the same way (1 = same direction, same meaning).

4.3 Probabilities, logprobs and softmax

- When the generator writes a word it has a probability for every possible next word. The probability of the word it chose tells us how **confident** it was. A **logprob** is the logarithm of that probability: 0 means certain, more negative means less confident.
- The checker outputs three probabilities (entailment, neutral, contradiction) that add up to 1.

Models first produce raw scores; **softmax** turns them into probabilities that are positive and add up to 1.

4.4 Transformers and attention

MiniLM and DeBERTa are **transformer** models, the same family as GPT. The key mechanism is **attention**: when the model processes a word, it decides how much every other word in the input matters for understanding it.

- A transformer has stacked **layers** (DeBERTa-base has 12), each with several **attention heads** (12). Different heads can focus on different relationships, e.g. matching numbers.
- **Self-attention** means the words of one input attend to each other. DeBERTa receives the chunk and the answer sentence joined into one input, so words in the sentence can attend to words in the chunk.

4.5 Classifiers vs generators

Kind	What it does	In HalluciGuard
Generator	Writes new text, word by word	qwen2.5:3b (writes the answer; also writes one kind of test attack)
Classifier	Reads text and picks a label	DeBERTa (entailment / neutral / contradiction)
Encoder (embedding model)	Reads text and outputs numbers	MiniLM (embeddings for search and for picking a sentence's source)

Using a classifier as the checker, instead of another generator, is deliberate: classifiers are fast, give the same answer every time, cannot invent text, and give probabilities we can put a threshold on.

> **Check yourself**
>
> Why does the system turn text into embeddings before it can find relevant chunks?
> *Answer:* computers compare numbers precisely. Embeddings let the system measure “closeness of meaning” as a number, so it finds chunks that mean the same thing as the question even when the words differ.

Part B

How the system works, step by step

> **The running example**
>
> **“How much new solar capacity was added globally in 2023?”**

5. The system at a glance

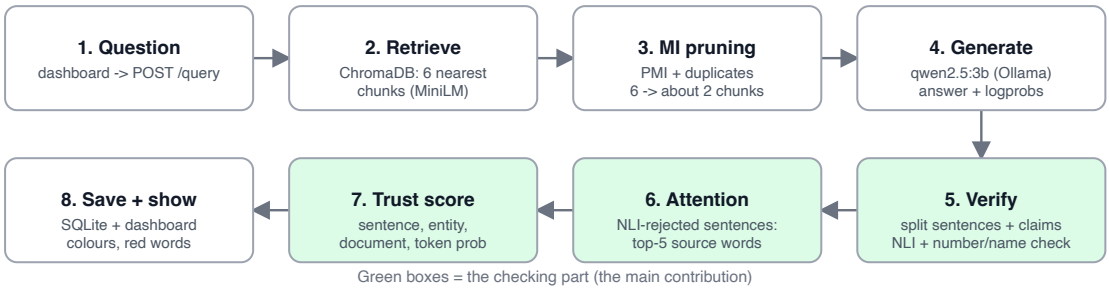


Figure 2. How one question flows through HalluciGuard. Steps 5-7 are the checking part.

Inside step 5, in order: fix split decimals ("42. 5" → "42.5"), split the answer into sentences, mark **filler** sentences, split compound sentences into **claims**, run **NLI** on every claim against every kept chunk, then run the **number and name check**, and decide each sentence's colour.

5.1 The main parts of the software

Part	Technology	Role
Backend (API)	Python + FastAPI	Receives requests, runs the pipeline, returns results; preloads models at startup
Dashboard	Streamlit	Input box, coloured answer, charts, three tabs
Vector database	ChromaDB	Stores chunk embeddings and finds the nearest ones
Relational database	SQLite (app.db)	Documents, chunks, questions, answers, sentence labels, scores, test results
Embedding model	all-MiniLM-L6-v2	384-number embeddings for search, and for choosing which chunk a sentence is about
Generator	qwen2.5:3b via Ollama	Writes the answer, returns token logprobs
Verifier	DeBERTa-v3-base NLI	Checks each claim against the sources
Language tools	spaCy (en_core_web_sm), NLTK	spaCy finds entities and splits compound sentences; NLTK splits sentences

The dashboard does no work itself; it calls the backend through the API. Logic lives in `app/core/` as plain functions; the API routes only call them.

5.2 The API endpoints

Method	Endpoint	What it does
POST	/documents	Adds a document: chunks it, embeds it and stores it
GET	/documents	Lists the stored documents

POST	/query	Full pipeline for a question; body: `text`, `k` (default 6), `inject_faults` (default false)
GET	/response/{id}	Fetches a stored result again
POST	/evaluate	Runs the 22 evaluation questions; options `retrieval_only`, `rescore`, `limit`
POST	/adversarial/run	Runs the adversarial test suite
GET	/adversarial/results	Detection rate per attack type and the false-positive rate
GET	/health	Checks that the backend is running

6. Step 0: Preparing the documents (ingestion)

Before anyone can ask a question, the documents are prepared once by running `scripts/seed_corpus.py`.

6.1 The corpus

The corpus is **6 documents (about 400 words each) on renewable energy**: solar, wind, hydropower, geothermal, energy storage, and policy and investment. They contain specific numbers and dates, which gives the checker something concrete to verify and the adversarial test something to corrupt. They were written for the project, which keeps the experiment controlled but means the results are not a public benchmark.

6.2 Chunking

Each document is cut into **word-based** windows of **250 words** that advance 210 words at a time, so neighbouring chunks **overlap by 40 words**. The cutting is not sentence-aware (it counts words, and rejoins them with single spaces). The 6 documents produce **12 chunks** (2 per document).

- **Precision:** finding one fact in one page is easier than in a whole book.
- **The checker's reading limit:** DeBERTa reads at most 512 tokens, chunk and sentence together; about 250 words of chunk leaves room for the sentence.
- **Shorter prompts:** the generator only gets the few chunks that matter.
- **Why overlap?** A fact at the boundary of two chunks could be cut in half; with 40 words of overlap it appears complete in at least one.

> **Be careful**
>
> Chunk size is a trade-off. Smaller chunks suit the checker better but can separate a sentence from the context it depends on (for example a pronoun from the name it refers to). 250 words is a reasonable middle; testing other sizes is future work.

6.3 Embedding and storing

Each chunk is embedded by MiniLM and stored in ChromaDB (cosine space); the chunk text, position and document details go to SQLite. Each document has an **authority** score between 0 and 1, used in the trust score. **In the current corpus all six documents have the same authority, 0.8** (the default), so this component carries no information yet (see section 13.3).

6.4 The database tables

Table	What it stores
documents	Title, source, authority score
chunks	Chunk text, its document, its position, its ID in ChromaDB

queries	Each question asked
responses	The full answer, model, **generator**, chunks retrieved and kept, overall trust, **logprobs_source**, **faults_injected** (set only in faulty mode)
response_sentences	Each sentence: NLI label and confidence, flagged or not, best chunk, top attention words, **wrong_spans** (the red words), **is_filler**
entities	Each name/number/date in an answer, and whether it was found in the sources
trust_scores	The four trust components and the final score
adversarial_tests	Each test: attack type, original and corrupted sentence, NLI label, detected or not

7. Step 1: Retrieval — finding the right chunks

The question is turned into an embedding with the same MiniLM model. ChromaDB then returns the **6 chunks closest to the question** (cosine similarity). These are the **retrieved** chunks.

7.1 Why embeddings instead of word search?

Meaning matters more than exact words. If the user asks “How much solar power was installed?” and a document says “new photovoltaic capacity added”, a word search finds almost nothing in common, but the embeddings are close.

7.2 The weakness of retrieval

Embeddings are deliberately “loose”. A chunk about “wind capacity added globally in 2023” is also close to our solar question. So the top 6 always contain related but useless chunks, and irrelevant context can confuse the generator. That is why the next step exists.

> **Check yourself**
> Why is a wind chunk a problem when the question is about solar?
> *Answer:* the generator might mix wind facts into the solar answer, a hallucination caused by bad context.

8. Step 2: MI pruning — removing useless chunks

Pruning looks at the 6 retrieved chunks and keeps only those that genuinely match the question. 6 chunks usually become about 2.

8.1 The question pruning asks

> **Key idea**
> "Does this chunk use the question's words **more than the other retrieved chunks do**?"

A word that appears everywhere ("energy") says nothing about which chunk is relevant; a word concentrated in one chunk ("solar" in a solar chunk) is a strong signal. This is measured with PMI.

8.2 PMI in plain words

PMI (pointwise mutual information) compares, for each word of the question, how often it appears *in this chunk* with how often it appears *across all the retrieved chunks*:

```
PMI(question, chunk) = average over question words t of
    ln( (P(t | chunk) + 0.000001) / (P(t | all retrieved chunks) + 0.000001) )
(counts are smoothed by adding 1 to every word count)
```

Question words are lower-cased, stop words ("the", "was"...) are removed and repeated words counted once. Equal frequency scores 0, more common in the chunk is positive, less common is negative.

> **Example**
> Question words: "solar", "capacity", "added", "2023". If "solar" is 6 times more common in Chunk A than on average, it adds ln 6 ≈ 1.8 to Chunk A's average. In a wind chunk "solar" never appears, giving a negative contribution. Words like "capacity" appear everywhere and contribute about 0.

8.3 The pruning rules

1. **Relevance cut-off:** go through the chunks from best to worst score and keep a chunk only if its score is at least **half of the best score**. (If every score is zero or negative, the cut-off is relaxed to twice the best score.)
2. **Duplicate removal:** also skip a chunk if its embedding similarity to a chunk already kept is above **0.85**.
3. **Safety net:** always keep at least **2** chunks; if the rules left fewer, top up with the next-best chunks, preferring non-duplicates.

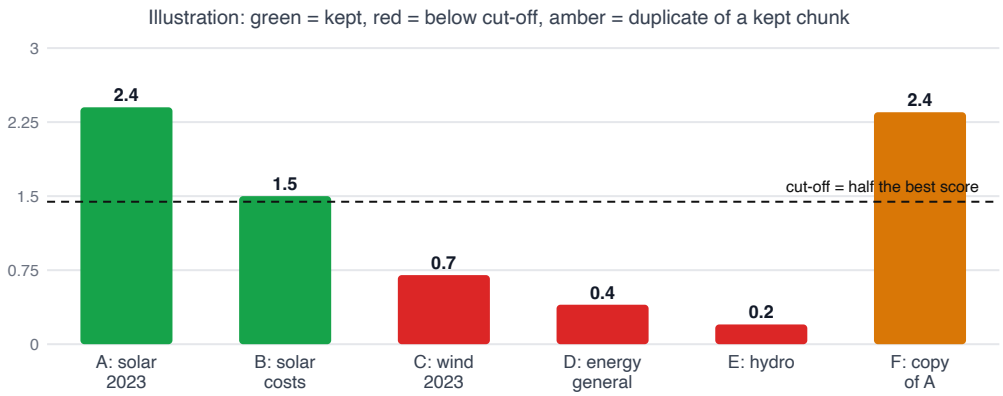


Figure 3. Illustration of pruning six retrieved chunks.

8.4 Why prune?

- **Fewer context-induced hallucinations:** the generator is not distracted by off-topic chunks.
- **Shorter prompts:** less to process.
- **Cleaner verification:** fewer chunks to compare each sentence against.

8.5 Why PMI and not a trained model?

Mutual information can also be estimated with neural networks, but those need training data and time. The PMI formula is closed-form, fast, and every score can be explained.

8.6 The result

Pruning drops about **62%** of retrieved chunks (6 → about 2) and precision rises from **0.33 to 0.74**. The ablation (section 18.3) also shows that **pruning improves the checker itself**: feeding all 6 chunks to NLI lowered recall from 0.84 to 0.67 and F1 from 0.71 to 0.65.

> **Check yourself**
>
> A wind chunk mentions “2023”, which is in the question. Why is it still dropped?
> *Answer:* “2023” appears in most retrieved chunks, so it adds little; and the chunk never mentions “solar”, which pulls its score below half of the best.

9. Step 3: Generation — the AI writes the answer

9.1 What the generator receives

The kept chunks are numbered and placed in a prompt with the question. A system instruction tells the model how to answer. The model runs at **temperature 0** (always the most likely next word), so the same prompt gives the same answer.

Mode	System instruction (verbatim)
Strict (default)	“Answer the question using only the numbered context passages below. Respond in 3-6 short sentences. If the context does not contain the answer, say so.”
Loose (test mode)	“Answer the question thoroughly in 4-6 sentences. The context passages below are a starting point; enrich the answer with additional background, specific figures, dates, named examples and comparisons from your own knowledge. Do not mention the context or say when information is missing.”

> **Example answer**
> “In 2023, about 345 gigawatts of new solar capacity was added globally. This information is directly stated in the context provided.”
> (Second sentence is *filler*: it makes no claim, see section 10.7.)

> **Be careful**
> The model is *told* to use only the context, but it doesn't always obey. This is exactly why verification is needed.

9.2 Logprobs: the model's confidence

The generator returns a logprob for every word it writes. Averaging them (and converting back with `exp`) gives the model's typical per-word probability, which becomes the **token-probability** part of the trust score. If a model returned no logprobs, this component is simply missing and the other weights are scaled up (section 13.6).

9.3 The cache

Every answer is saved on disk, keyed by the model, the system instruction and the exact prompt. If the same prompt is sent again, the saved answer is reused. This makes re-running evaluations fast. (The prompt is part of the key, so strict and loose answers never mix.)

9.4 Why a local model (Ollama) and not an online API?

The project first used Google's Gemini API, then **removed it completely**. Two problems appeared: the Gemini models available rejected logprobs (“Logprobs is not enabled”), and the free tier allowed only a small number of requests per day, which made repeated evaluation impossible. With Ollama running **qwen2.5:3b** (a 3-billion-parameter model):

- there are no usage limits and nothing leaves the computer;
- per-token logprobs are returned, so all four trust components work;
- generation takes about **2.5 seconds**; a whole `/query` (retrieval, generation, NLI) takes about **8 seconds**, of which NLI is about 3.
- the API preloads the models at startup and asks Ollama to keep the model in memory (30 minutes), so even the first query is fast.

> **Key idea**
> A side effect worth knowing: small local models hallucinate somewhat more than large online ones. For a project about *detecting* hallucinations that is useful, because the checker has real errors to catch. In strict mode though, this model is mostly faithful (section 18), which is why the test modes exist.

10. Step 4: NLI verification — checking each sentence

This is the heart of the project. The answer is split into sentences (NLTK), compound sentences are split into smaller claims, and each claim is checked against each kept chunk using **NLI**.

10.1 What NLI is

NLI (natural language inference) answers one question about two texts: **"If Text A is true, is Text B also true?" Premise** (Text A) = a chunk from the documents. **Hypothesis** (Text B) = the claim being checked.

Label	Meaning	Example (premise: "In 2023 the world added about 345 GW of new solar capacity.")
Entailment	The premise supports the hypothesis	"More than 300 GW of solar was added in 2023."
Neutral	The premise doesn't say either way	"Solar panels are popular in India."
Contradiction	The premise says the opposite	"Only 100 GW of solar was added in 2023."

"More than 300" is not written in the premise, but 345 is more than 300, so it is supported. **NLI judges meaning, not copied words.**

10.2 Why the chunk is the premise and the sentence is the hypothesis

We want to know whether the *sources* support the *answer*, so the source plays "what is known to be true" and the answer is the claim under test. Swapping them would ask whether the answer supports the whole chunk, which it almost never does.

10.3 The model: DeBERTa

The checker is **DeBERTa-v3-base**, specifically `MoritzLaurer/DeBERTa-v3-base-mnli-fever-anli`, a Microsoft transformer further trained for NLI on MNLI (many kinds of text), FEVER (claims checked against Wikipedia) and ANLI (hard pairs written to fool models). It learned patterns such as "not" flipping meaning, or different numbers for the same thing signalling contradiction. Nobody wrote these as rules. The model is used **pretrained, not fine-tuned** (section 20).

10.4 What happens inside, step by step

```
[CLS] premise text [SEP] hypothesis text [SEP]      (at most 512 tokens; the chunk is cut first if too long)
```

The model reads both texts at the same time, so every word can be related to every other word (this is how it notices that "345 GW" in the chunk matches "345 gigawatts" in the sentence). It outputs three raw scores, and softmax turns them into probabilities, for example entailment 0.94, neutral 0.05, contradiction 0.01 (illustrative).

> **Be careful**
> Different NLI models order their labels differently. The code reads label names from the model's configuration (`model.config.id2label`) instead of assuming. For this model the configuration lists 0 = entailment, 1 = neutral, 2 = contradiction.

10.5 Checking a whole answer: the sentence × chunk table

An answer has several sentences and there are usually 2 kept chunks, so every claim is checked against every kept chunk. The chunk with the highest entailment probability is the claim's **best chunk**. A fact only needs to be supported by *one* source.

	Chunk 1	Chunk 2
Sentence 1 → green	**0.94 (best)**	0.10
Sentence 2 → green	0.20	**0.85 (best)**
Sentence 3 → red (best 0.30 < 0.5)	0.15	**0.30 (best)**

Figure 4. Entailment probability for every sentence-chunk pair (illustrative). The best chunk decides.

10.6 Atomic claims: splitting compound sentences

Many answer sentences contain two facts joined by “and”. If one is wrong and one is right, a whole-sentence check can be fooled. So compound sentences are split, using spaCy's grammar analysis, at coordinated verbs. A clause that has no subject, or only a pronoun like “it”, borrows the main clause's subject so it can stand alone:

> **Example**
> “The Three Gorges Dam has a capacity of 22,500 megawatts *and it* typically generates between 85 and 100 terawatt-hours per year.”
> → claim 1: “The Three Gorges Dam has a capacity of 22,500 megawatts.”
> → claim 2: “The Three Gorges Dam typically generates between 85 and 100 terawatt-hours per year.”

The split only happens when it is safe: at most 4 claims, each at least 4 words, otherwise the sentence stays whole. A sentence takes the verdict of its **weakest claim** (the lowest best-entailment); it is flagged if any claim is flagged. This does **not** fix pronouns that refer to a previous sentence (“It was completed in 2012.”).

> **Be careful**
> In the ablation (section 18.3) claim splitting did **not** measurably improve F1 (0.73 vs 0.74 for sentence-level NLI; the intervals overlap). Present it as a reasonable design that was measured and gave no clear gain.

10.7 Filler sentences

Some sentences make no claim: “This information is directly stated in the context provided.” or “The context does not contain the answer.” A sentence is **filler** when, after removing [1]-style citations, it has **no digit**, is **at most 25 words**, and mentions a reference word (context, passage, source, document...)

together with a meta word (stated, provided, mentioned, according, found, does not contain...). Filler is shown in grey italics, is **never flagged**, and is excluded from the sentence score, the document score and the evaluation counts. A **claim sentence** is any sentence that is not filler.

10.8 The decision rule (exactly how a sentence gets its colour)

Condition (checked in this order)	Result
Sentence is filler	*grey* : never flagged
The weakest claim's best entailment is **below 0.5**	red (flagged; shown with a source chunk and attention words)
The number/name check found wrong parts (section 11)	amber sentence with **bright red** words (flagged)
Best entailment ≥ 0.5 and the label is entailment	green (supported)
Best entailment ≥ 0.5 but the label is neutral or contradiction	amber

Three details: (0) if a sentence is rejected by NLI *and* has wrong parts, the dashboard shows it amber with the bright red words; (1) the label is "contradiction" whenever *any* chunk gives contradiction above 0.5, but a sentence only turns red if its *best* entailment is below 0.5; (2) the 0.5 threshold is a design choice, not learned. The ablation sweep (section 18.3) found that thresholds from 0.3 to 0.9 all give F1 of about 0.71-0.72, so the result is not sensitive to it.

10.9 Why sentence by sentence?

If the whole answer were checked at once, five correct sentences and one wrong one would still mostly "agree" with the sources and the error would hide. Checking each sentence separately finds **exactly which claim** is unsupported (sentence-level hallucination localisation).

10.10 Similar words are not the same meaning

> **A classic trap**
>
> Premise: "**Wind** capacity grew in 2023." Hypothesis: "**Solar** capacity grew in 2023." Five of six words match, but the premise says nothing about solar. The correct label is **neutral**. A word-matching checker would wrongly say "supported"; this is why NLI is used.

10.11 Where NLI makes mistakes

Weak spot	Example	Why it happens
Pronouns	"It was completed in 2012."	What "it" refers to is in another sentence
Long premises	One relevant line inside a 250-word chunk	Trained mostly on short premises; long text dilutes the evidence
Exact numbers	A wrong figure got entailment 0.97; "In 2030, about 172 GW..." got 0.68	It learned meaning, not arithmetic; the chunk is full of similar numbers

Loosely related chunks	A policy chunk 'entails' a sentence about solar because it also mentions 2030 and gigawatts	The best chunk can be a weakly related one
Subtle rewrites	A fluent sentence with one detail quietly changed	Obvious negations are easy; small shifts are hard

These weaknesses explain most of the errors in the results, and the number and name check (next section) was added to address the numeric ones. The ablation also shows that **contradictions and miscopied facts** are the hardest category (recall 0.44 on 9 examples).

> **Check yourself**
>
> Premise: “Wind capacity grew 10% in 2022.” Hypothesis: “Wind capacity did not grow in 2022.” Which label?
> *Answer:* contradiction.

11. Step 5: The number and name checker (span check)

Testing showed that NLI is weak with exact numbers: a sentence with a wrong figure once got entailment 0.97. Wrong numbers are among the most common and most harmful hallucinations, so a second, simpler check was added. It also lets the dashboard mark **exactly which words** are wrong instead of colouring the whole sentence.

11.1 What it checks

- **Numbers (including years):** every standalone number in the sentence must appear among the numbers of the **source passage the sentence is about**. A number that does not is marked bright red.
- **Names:** every person, organisation, place, nationality, facility, product, event or law name (found by spaCy) must appear somewhere in **any** kept chunk. A name that does not is marked bright red.

11.2 Exactly how matching works

Step	Rule
Which number counts	Standalone numbers like 345, 22,500, 7.5, 2023. **Skipped:** numbers inside words (COP28), citations [1], and numbers followed by letters (2030s, 5th); number words (“three”)
Normalisation	Only commas are removed: 22,500 = 22500. **No** unit or magnitude conversion (1.6 thousand, 0.345 TW) and no rounding (42.5 ≠ 42.50)
Which passage (the “reference chunk”)	The kept chunk the sentence is *about*: highest embedding similarity to the number-free sentence, plus a small boost for how often the sentence's content words occur in the chunk. Chosen by meaning, **not** by NLI, because NLI can prefer a loosely related chunk (the 2030 example)
Rescue rule	A number found only in the NLI-preferred chunk is still accepted if NLI entailment there is at least 0.9
Names	Hedge words (about, roughly, the...) are stripped; the name must appear (lower-cased substring) in any kept chunk
Skipped entirely	Filler sentences; dates and quantities as entities (their numbers are covered by the number rule); relative dates like “the end of that year”

A sentence with any wrong part becomes **flagged**, is shown amber with the wrong words in bright red, and its displayed source becomes the reference chunk.

> **Example (faulty mode)**
> Original: “In 2023, about 345 gigawatts of new solar capacity was added globally.”
> Corrupted: “ In **2013**, about **690** gigawatts of new solar capacity was added globally. ”
> NLI said *entailment* (0.80), but neither 2013 nor 690 is in the Solar chunk, so both are marked.

11.3 Its limits, and why it is still useful

The check confirms that a number **exists** in the source passage, not that it is attached to the right fact. The two checks cover each other:

Check	Good at	Weak at
NLI	Meaning of the whole sentence	Exact numbers
Number and name checker	Exact numbers and names	Context and meaning; units and rounding

Be careful

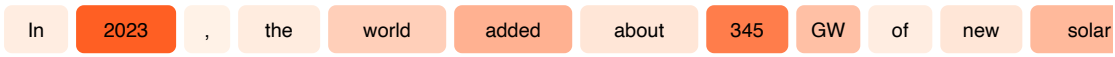
The ablation (section 18.3) shows the span check alone has precision 0.57 but low recall (0.22), and adding it to NLI did **not** raise overall F1 on the labelled set, which consists mostly of invented details and commentary rather than changed numbers. Its value is explanation (marking the exact words) and catching changed numbers, which the adversarial/faulty tests show.

What to say

“NLI judges the meaning of the sentence; the number checker makes sure the specific numbers and names exist in the sources. Each covers the other's weakness, and the second one tells the user exactly which words to distrust.”

12. Step 6: Attention attribution — where was the checker looking?

For sentences that **NLI itself rejected** (best entailment below 0.5), the system shows the **top 5 words in the best source chunk** that the checker paid most attention to. (Sentences flagged only by the number check do not get this view, because NLI accepted them.)



Flagged sentence: “In 2017, the world added about 345 GW of new solar capacity.” Darker = more attention (illustrative).

Figure 5. Illustrative attention view: darker source words received more attention from the flagged sentence.

12.1 How it is computed

1. Run DeBERTa on (best chunk + flagged sentence) and ask it to also return its attention weights.
2. Average the weights over the **last 4 layers** and **all 12 heads**.
3. Keep only attention **from the sentence's tokens to the chunk's tokens**, and add it up for each chunk token.
4. Merge word pieces back into whole words (“giga” + “watts” → “gigawatts”).
5. Remove special tokens, punctuation and common words (“the”, “of”).
6. Return the top 5 words, with weights scaled to add up to 1 over those five.

12.2 The correct way to describe it

> **Be careful**
>
> Attention shows **where the checker focused**. It does **not** prove what **caused** its decision; research shows attention weights and the true reasons don't always match. Say “the source words the checker attended to most”. DeBERTa uses **self-attention** over the joined pair, not cross-attention. The optional check from the plan (hide the top 5 words and re-run NLI to see whether the verdict changes) was **not built**.

13. Step 7: The hierarchical trust score

The trust score summarises how reliable an answer is as one number from 0 to 1. It combines four signals:

Component	Weight	Question it answers	Level
Sentence	0.35	Do the sources support the claims? (from NLI and the wrong-part check)	Sentence
Entity	0.25	Do the names, numbers and dates in the answer appear in the sources?	Entity
Document	0.20	How trustworthy are the documents behind the supported sentences?	Document
Token probability	0.20	How confident was the generator while writing?	Token

```
trust = 0.35 x sentence + 0.25 x entity + 0.20 x document + 0.20 x token probability
```

13.1 Sentence score

Over the **claim sentences** (filler excluded): the average best-entailment probability × (1 – fraction of sentences flagged). A flagged sentence lowers both factors. If there are no claim sentences the score is 0.

13.2 Entity score

spaCy finds every entity in the whole answer (names, numbers, dates, percentages...). Relative dates (“the end of that year”) are skipped, hedge words (“approximately”, “roughly”, “the”) are removed, duplicates counted once. An entity is **supported** if its lower-cased text appears (as a substring) in the kept chunks. The score is the fraction supported; **no entities means 1.0**.

13.3 Document score

The average authority of the documents behind the non-flagged claim sentences; 0 if none. **All six documents currently have authority 0.8**, so this component only varies with how many sentences are supported, and carries no extra information about source quality (the AUROC is 0.46, section 18.3).

13.4 Token-probability score

`exp(average logprob)` over the generator's words: its typical per-word probability. It describes the *original* answer, so in faulty mode it is dropped.

13.5 A worked example

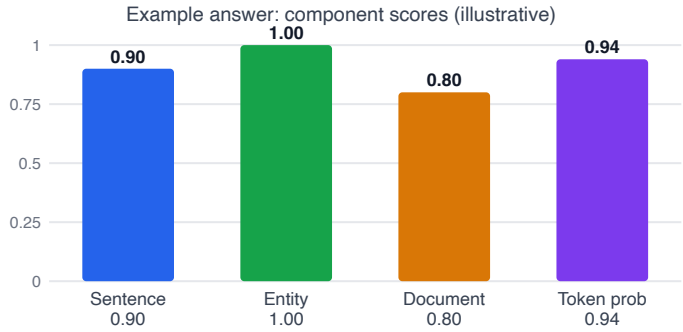


Figure 6. Component scores for an example answer. Weighted sum: 0.35x0.90 + 0.25x1.00 + 0.20x0.80 + 0.20x0.94 = 0.913.

13.6 When a component is missing: renormalisation

If a component has no data, its weight is removed and the others are scaled up to add to 1. Without token probability the other weights add to 0.80, so they become 0.35/0.80 = 0.4375, 0.25/0.80 = 0.3125 and 0.20/0.80 = 0.25.

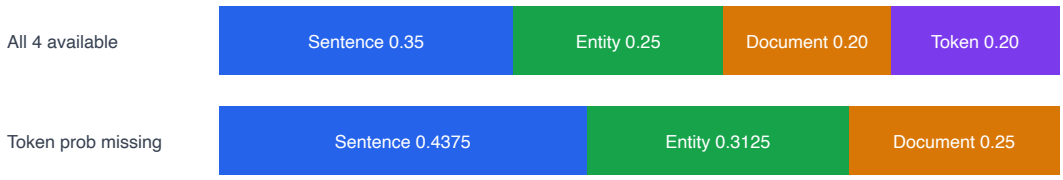


Figure 7. Renormalisation keeps the total weight at 1.

> **Example (faulty mode, no token probability)**
>
> A corrupted answer had sentence 0.76, entity 0.50 and document 0.80: 0.4375×0.76 + 0.3125×0.50 + 0.25×0.80 = 0.333 + 0.156 + 0.200 ≈ **0.69**.

13.7 Why “hierarchical”, and why combine four signals?

Each signal works at a different level: tokens (words), entities (specific facts), sentences (whole claims) and documents (source quality). Each alone has blind spots: NLI alone gave a wrong number 0.97 entailment; entity matching alone cannot tell whether a number is attached to the right fact; document authority says nothing about whether a claim matches.

> **Key idea**
>
> **Confidence is not correctness**, but it is informative. On our labelled answers the generator's token probability separated good from bad answers best (AUROC 0.98, section 18.3). Caveat: loose-prompt answers are both less confident *and* more often unsupported, so part of that signal is the prompt mode.

13.8 How the weights were chosen

The weights were **set by design**, not learned. Sentence support gets the most weight as the most direct evidence. The ablation shows token probability would deserve more weight than 0.20 on this data, and the document component deserves less; learning the weights from labelled data is future work.

> **What to say**
> "The weights are designed, not learned. Our own analysis suggests token probability is stronger than its weight implies, and the document component is uninformative while all authorities are equal. I would learn the weights from more labelled data."

13.9 Known edge cases

- **One claim, flagged:** the sentence and document scores become 0 and the trust score drops to about 0 (we saw 0.00 for a faulty-mode answer with one claim). A smoother formula would help.
- **Only filler:** sentence and document scores are 0, so trust is low.

14. Step 8: Saving and showing the result

Everything is saved in SQLite: the question, the answer, retrieved and kept chunks, each sentence's label, confidence, best chunk, attention words and wrong parts, the entities, and all trust components. `GET /response/{id}` fetches it again.

14.1 The dashboard

Tab	What it shows
Query	Input box and a **Faulty mode** checkbox. After a question: model used, chunks retrieved and used, the coloured answer, in faulty mode a “Caught / Missed” line per injected error, an expander per flagged sentence (NLI label, wrong parts, source chunk, attention words), the entity list with ✓/✗, the trust score and a bar chart of its components
Robustness	Adversarial results: detection rate per attack type and the false-positive rate; a button to run the suite
Evaluation	Evaluation results: precision, hallucination rate, average prune ratio, average trust

What you see	Meaning
green sentence	Supported: best entailment at least 0.5, label entailment
amber sentence	Neutral, or flagged because it contains wrong parts
bright red words	A number or name the sources do not contain (hover shows the reason)
light red sentence	Rejected by NLI (best entailment below 0.5)
grey italic sentence	Filler: no claim to check

Part C

Testing the system and understanding the results

15. The adversarial test: does the checker actually work?

To know whether a checker is good, you must give it sentences **you already know are wrong** and see whether it catches them. That is what the adversarial test does ("adversarial" = deliberately trying to break the system).

15.1 Normal use vs testing

	Normal use (every question)	Adversarial test (separate script)
Input	The generator's real answer	Sentences we broke on purpose
Do we know the right answer?	No: that is what we're trying to find out	Yes: we know exactly which sentences are wrong
Purpose	Find hallucinations in answers	Measure how good the checker is

> **Key idea**
> In normal use, nothing fake is added. The adversarial test is a separate measurement of the checker itself, like giving a teacher an answer sheet with known mistakes to see how many they find.

15.2 How it works

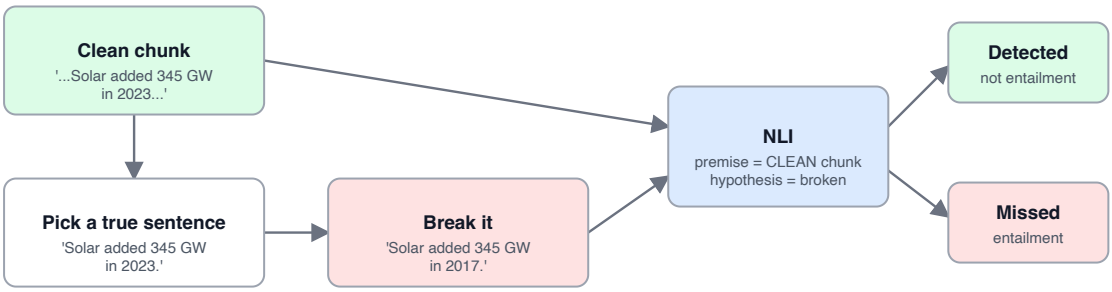


Figure 8. The adversarial test design.

1. **Which sentences:** chunks are shuffled with a fixed seed (42) and up to 20 are taken; the corpus has only 12, so all 12 are used. From each chunk up to 2 sentences are picked (sentences with a number first), each at least 5 words and containing a number or a verb: 24 sentences.
2. **Corrupt it** with each attack below; the unchanged sentence is the control.
3. **Run NLI** with the **original clean chunk** as premise and the corrupted sentence as hypothesis.
4. **Detected** means NLI's top label is anything other than entailment. Each run replaces the previous results: 24 sentences × 5 variants = 120 tests.

15.3 The attack types

Attack	Exact rule	Example
Negation	If “not” or “never” is present, remove the first; otherwise insert “not” after the first is/was/are/were/has/have/had/can/will/does/did; otherwise prefix “It is not true that”	“Solar added 345 GW.” → “Solar did not add 345 GW.”
Stale fact	Every year (1500-2099) shifts by the same random 3 to 10 years (earlier or later); every other number is multiplied by a random choice of 0.5, 0.6, 0.75, 1.5 or 2	“...in 2023” → “...in 2017”; “345 GW” → “172 GW”
Noise	Half the time (if the sentence has a name and another is available) swap one name for a random name from the corpus; otherwise append one of four unrelated clauses	“...in China” → “...in Brazil”
LLM contradiction	Ask qwen2.5:3b (temperature 0): “Rewrite this sentence so it remains fluent and plausible but states something factually different. Output only the sentence.”; results are cached	A fluent sentence with one detail changed

The first three are **rule-based**; the fourth uses the generator for subtler attacks. The “automated adversarial agent” in the proposal is this script: a program with a policy for choosing sentences and attacks, not a trained reinforcement-learning agent.

15.4 The control set: why it is essential

The unchanged true sentences are also checked; they should pass. When the checker flags an unchanged sentence, that is a **false positive**.

> **Key idea**
>
> Without a control set, a checker that flags *everything* would score 100% detection and look perfect. The control set shows whether the checker is accurate or just suspicious of everything.

15.5 Why not corrupt the documents and ask normal questions?

That measures nothing: the generator would faithfully repeat the corrupted text, and NLI would correctly say the (corrupted) source supports it. Comparing corrupted sentences against the **clean** source is what makes the test meaningful.

15.6 Interpreting the results

Important: this test runs **NLI alone**. It does not use claim splitting or the number and name check, so it measures the base checker, not the whole pipeline (the labelled-set evaluation in section 18.3 measures the full pipeline).

LLM-written contradictions were detected only **21%** of the time. Two explanations, and both may be true:

- **The attacks are weak:** the small 3B model often wrote near-paraphrases (“approximately”, a small year change) instead of real contradictions; a “contradiction” that does not contradict is correctly passed by the checker.

- **The checker is weak on subtle changes:** fluent rewrites with a small shift in meaning are genuinely harder for NLI than obvious negations.

> **Be careful**
> Until someone reads the attack sentences and marks which are real contradictions, you cannot say how much of the 21% is due to each cause. Say: “Detection on LLM-written attacks was 21%; many rewrites were near-paraphrases, so this test is weak.”

16. Test modes: faulty mode and loose mode

The checker's value is only visible when an answer contains errors. Two clearly labelled test modes create errors on purpose:

Mode	Exactly what it does	How to switch it on
Faulty mode	After the generator answers, corrupts **1 sentence (2 if the answer has 4 or more)**: sentences with numbers first (all their numbers are changed, using the stale-fact rule), otherwise a negation. Then the normal checks run. The response is tagged, its token probability is dropped (the logprobs describe the original text), and it is excluded from evaluation	**Dashboard:** the “Faulty mode” checkbox on the Query tab (API: `inject_faults: true`)
Loose mode	Replaces the “answer only from the context” instruction with one that invites the model to add outside background, figures and examples (verbatim in section 9.1)	**Environment variable** `PROMPT_MODE=loose` before starting the backend. There is no dashboard switch, and the mode is not stored with the answer

> **Why test modes are legitimate**
>
> This is not cheating. The project's purpose is *detecting* hallucinations, and a detector can only be tested on answers that contain some. The modes are labelled openly, and normal mode stays strict, as real RAG systems are. In the report: “We used a loose prompt to create more hallucinations for testing.”

In loose mode many red sentences may be *true* facts from the model's general knowledge, or plain commentary. By the project's definition (section 3) they are still unsupported, so flagging them is correct.

17. How the system is measured: the metrics

17.1 The evaluation set

22 questions about the documents, each with a reference answer and the list of documents that contain it. The evaluation (`POST /evaluate` or `scripts/run_eval.py`) runs them all through the system. Cheaper modes: `retrieval_only` (retrieval and pruning metrics, no generation), `rescore` (re-check the stored answers, no new generation) and `limit` (first N questions).

17.2 Metric definitions

Metric	Plain meaning	Better is
Precision@6	Of the 6 retrieved chunks, the fraction from a relevant document	Higher
Precision after pruning	Of the chunks kept after pruning, the fraction that are relevant	Higher
Prune ratio	The fraction of retrieved chunks that pruning removed	—
Hallucination rate	Of all **claim sentences** (filler excluded), the fraction flagged	Lower (for the generator)
Average trust	Mean trust score across the answers	Higher
Detection rate	Of the corrupted sentences of one attack type, the fraction the checker caught	Higher
False-positive rate	Of unchanged control sentences, the fraction wrongly flagged	Lower
Precision / recall / F1	On the labelled set, treating “unsupported” as the target: precision = of flagged sentences, how many were really unsupported; recall = of the really unsupported, how many were flagged; F1 = their balance	Higher
95% confidence interval	A range for F1 from 1000 random re-samplings (bootstrap): how much the number could move by luck	—
AUROC	Chance that a randomly chosen bad answer scores worse than a randomly chosen good one (0.5 = guessing, 1 = perfect)	Higher
Latency	Time to generate one answer (about 2.5 s); a full query is about 8 s	Lower

17.3 Why precision@6 can never exceed 0.33 here

Each document produces only 2 chunks. If the answer is in one document, at most 2 of the 6 retrieved chunks can come from it: the best possible precision is 2/6 = 0.33. The measured 0.33 is a **ceiling of the setup, not a failure of retrieval**. Recall@k and hit rate would suit this setup better.

18. The results

> **About the numbers**
> All figures below were measured on 2026-10-11 with the code described in this guide, except where marked. The 22-question evaluation (8.7% hallucination rate) was run **before claim splitting was added** and has not been re-run with it; re-run `python scripts/run_eval.py` if you want the number for the final code.

18.1 Results table

What	Result	What it means
Precision@6 (before pruning)	0.33	At the ceiling of the setup (2 of 6)
Precision after pruning	0.74	About 3 in 4 kept chunks are relevant
Chunks removed by pruning	62%	6 chunks → about 2
Hallucination rate (22 questions, strict)	8.7% (2 of 23 claim sentences)	Very few flagged; the sample is small, say “2 of 23”
Average trust	0.90	Most answers well supported
Average token probability	0.94	The generator is confident
Time per answer	about 2.5 s (generation); about 8 s end to end	Fast enough for live use
Negation / stale fact / noise detected (NLI alone)	100% / 100% / 100%	Clear errors are caught
LLM-written contradictions detected	21%	Weak test: many near-paraphrases
False-positive rate (control, NLI alone)	16.7%	About 1 in 6 true sentences wrongly flagged
Automated tests	43 passing	pytest, run on a temporary database

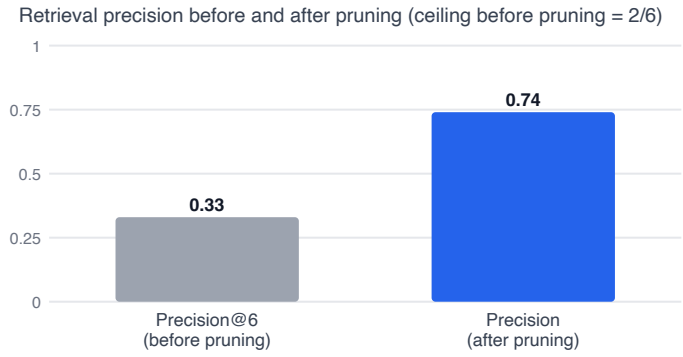


Figure 9. Retrieval precision before and after pruning.



Figure 10. Adversarial results.

18.2 What the results show

- **Pruning works:** it removes most chunks while sharply raising the share of relevant ones, and it improves the checker (18.3).
- **The checker catches clear errors reliably:** negations, changed numbers and noise were all detected.
- **Subtle errors are harder:** LLM-written rewrites (weak test) and contradictions in the labelled set (recall 0.44).
- **The checker is somewhat over-cautious:** it flags about 1 in 6 unchanged true sentences (adversarial control) and about 1 in 5 supported sentences in the labelled set (false-positive rate 0.23).
- **The local generator is mostly faithful in strict mode** (2 of 23 sentences flagged), which is why the test modes are needed for demonstrations.

18.3 Labelled-set evaluation and ablation

To measure the full pipeline on *natural* answers, 92 real answers from the generator (strict and loose prompt) were split into 215 sentences, and each sentence was labelled **supported** (121), **unsupported** (55), **filler** (30, excluded) or **ambiguous** (9, excluded). The 176 usable sentences were run through variants of the pipeline.

> **Be careful: who made the labels**
>
> The labels were made by an **AI assistant** reading each sentence against the source documents, **not by a human**. Spot-check `data/eval/labeled_sentences.json` before presenting these numbers as ground truth, and say so if asked. Sentences from the same answer are correlated, so the confidence intervals are optimistic.

Variant	Precision	Recall	F1 (95% CI)	False-positive rate
NLI only (sentence level)	0.68	0.82	**0.74** (0.65-0.83)	0.17
Number/name check only	0.57	0.22	0.32 (0.17-0.46)	0.07
NLI + number/name check	0.65	0.84	0.73 (0.64-0.81)	0.21
NLI + claim splitting	0.65	0.82	0.73 (0.63-0.81)	0.20

Full pipeline (claims + check + filler) — the app	0.62	0.84	0.71 (0.62-0.80)	0.23
Full pipeline **without** MI pruning (6 chunks)	0.63	0.67	0.65 (0.54-0.74)	0.18

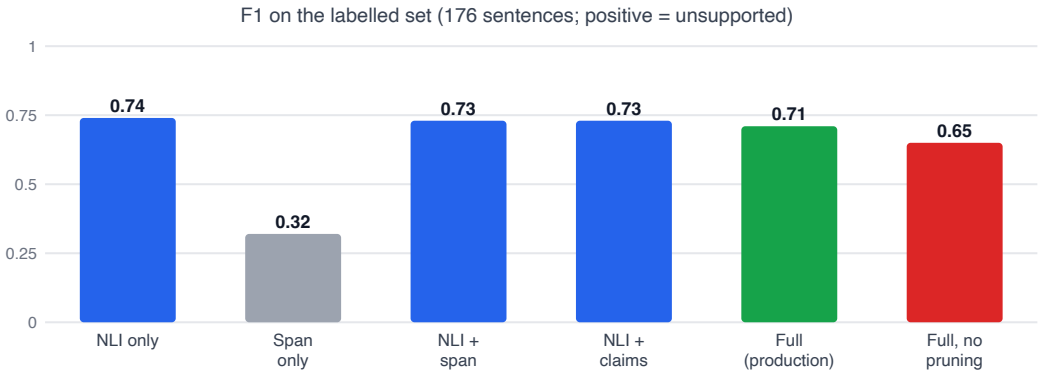


Figure 11. F1 by pipeline variant.

- **MI pruning clearly helps:** recall 0.84 vs 0.67 (F1 0.71 vs 0.65). Giving NLI all 6 chunks dilutes the evidence.
- **Claim splitting and the number/name check did not measurably improve F1.** All NLI variants lie within each other's intervals (0.71-0.74). They trade a little precision for a little recall. They remain useful for explanation (marking the exact wrong words) and for catching changed numbers, which this labelled set (mostly invented details and commentary) does not reward. Say this honestly.
- **Entailment threshold:** from 0.3 to 0.9, F1 stays at 0.71-0.72 (precision 0.63 → 0.57, recall 0.82 → 0.95), so 0.5 is not critical; higher thresholds trade precision for recall.
- **Recall by error type** (full pipeline): unsupported commentary 0.93 (27 sentences), invented specifics 0.89 (19), **contradictions or miscopied source facts 0.44 (only 9)**: the hardest case.

Answer-level trust analysis (83 answers, 33 containing an unsupported sentence): AUROC of each trust component for spotting bad answers:

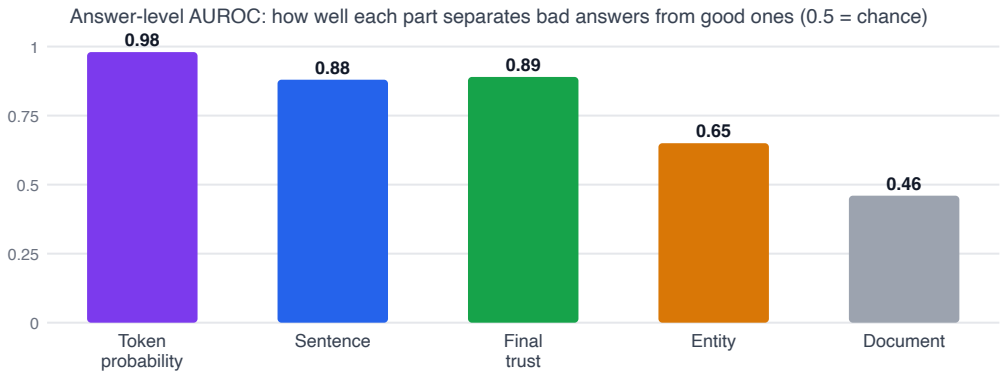


Figure 12. AUROC per trust component.

> **Be careful**
> Token probability (0.98) looks best, but loose-prompt answers are both less confident and more often unsupported, so part of that is the prompt mode. The document component (0.46, chance level) is uninformative because all documents have the same authority.

18.4 The pronoun problem in detail

Many false positives come from sentences that start with a pronoun. Example: the chunk says “The Three Gorges Dam ... It was fully commissioned in 2012” and the sentence “It was completed in 2012” is checked on its own. Without the name it refers to, the model is unsure what is claimed and gives low entailment. Claim splitting only helps *inside* a sentence. A “pronoun retry” (re-checking a flagged pronoun sentence together with the previous sentence) was **not built**; checking against smaller pieces of each chunk is the other fix.

18.5 Things to double-check before presenting

- The numbers in the tables match the latest `README.md` and `FACTS.md`. (The adversarial false-positive rate was 12.5% on an earlier run and 16.7% on the latest; sampling changed slightly when a year-handling bug was fixed.)
- The hallucination rate of 8.7% is 2 of 23 **claim sentences** (filler excluded) over 22 answers, and was measured before claim splitting.
- The labelled-set numbers rest on **AI-made labels** (18.3).
- Faulty mode works in the dashboard; loose mode needs the `PROMPT_MODE=loose` environment variable.
- The pronoun retry is **not** implemented; the threshold sweep **is** done (18.3).

Part D

Defending the project: limitations, decisions and questions

19. Limitations

Every system has limits. Naming them before a professor does shows real understanding, and professors reward it.

Limitation	Explanation	What could be done
Supported, not true	If a source document is wrong, a sentence copying it is marked supported	Combine with trusted, curated sources; inherent to the faithfulness approach
Weak on exact numbers (NLI)	NLI gave a wrong figure 0.97 entailment; a policy chunk 'entailed' a wrong solar figure	Number/name check (added); units, rounding and magnitude handling; a numeric reasoning step
Contradictions are the weakest case	Recall 0.44 on 9 labelled examples of facts miscopied from the source	A second verifier; sentence-level evidence; more labelled data
False positives	Adversarial control 16.7%; labelled set 23% of supported sentences flagged. Causes: pronouns, long chunks, number formats	Check against 2-3 sentence snippets; pronoun retry
Claim splitting and span check show no F1 gain	On the labelled set all NLI variants are within the same interval (0.71-0.74)	Evaluate on a set with more changed-number errors
Labels made by an AI assistant	The labelled set was not checked by a human; same-answer sentences are correlated	Human re-labelling of a sample; more annotators
Trust weights designed, not learned	Token probability looks stronger and documents weaker than their weights imply	Learn weights from labelled answers
Document authority is constant	All six documents have authority 0.8, so the document score carries no information	Give documents real, differing authority values
Single-claim answers	One flagged claim drives trust to about 0	A smoother formula
Small, self-written corpus	6 documents, 12 chunks, 22 questions; results indicative only	A larger public dataset
Weak attack generator	The 3B model writes near-paraphrases instead of contradictions	Manual review of attacks; a larger model for attacks
Adversarial test uses NLI alone	It does not exercise claim splitting or the number/name check	Run the attack set through the full pipeline
Number/name matching is simple	Substring matching, no units or rounding; a number can match in the wrong context within the reference chunk	Structured claim extraction (who, what, how much)
Attention is explanatory, not causal	Attention is not the reason for a decision; the masking check was not built	Masking check; other attribution methods
Loose mode is an environment variable	Not selectable in the dashboard and not stored with the answer	Add a toggle and record the mode

20. Design decisions and the alternatives considered

Professors often ask “why did you choose X instead of Y?”. Each decision has a short, honest reason.

Decision	Alternative	Why this choice
NLI classifier as the checker	Ask another LLM to judge	Runs locally, fast, same result every time, gives probabilities for a threshold; an LLM judge is slower, less predictable and can hallucinate itself
Pretrained DeBERTa	Fine-tune the NLI model	Fine-tuning needs a large labelled dataset, a GPU and a strict train/test split; the proposal specified a pretrained model. (Nothing in this project is fine-tuned.)
Local model via Ollama (Gemini removed)	Gemini or another online API	No daily limits, returns logprobs, private, reproducible
PMI formula for pruning	A trained mutual-information estimator	No training, fast, every score is explainable; the ablation shows pruning helps
Sentence-level checking, split into claims	Check the whole answer at once	Finds exactly which claim is wrong (claim splitting itself gave no measurable F1 gain)
Number/name check as a second verifier	Trust NLI alone	NLI is blind to exact numbers; the check also gives the exact words to mark
Reference chunk chosen by meaning	Use the chunk NLI likes best	NLI can prefer a loosely related chunk that happens to contain the same year
Filler sentences excluded	Score every sentence	“This is stated in the context” makes no claim and would inflate or distort the scores
Best chunk per sentence	Require support from all chunks	A fact only needs to be in one source
Rule-based + LLM attacks	A trained reinforcement-learning attacker	Feasible in the time available; transparent rules
ChromaDB + SQLite	Larger databases (Postgres)	Zero setup, file-based, enough for this scale
Streamlit dashboard	A custom web app (e.g. React)	Charts and highlighted text with very little code; the plan lists React as not used
Temperature 0, cached answers	Random sampling	Reproducible answers for evaluation

21. Future work

- **Sentence-level evidence:** check each claim against the 2-3 most relevant source sentences instead of a whole 250-word chunk (likely the biggest single improvement for precision and for explanation).
- **Human-verified labels:** have a person re-label at least a sample of the labelled set, and extend it.
- **Learn the trust weights** from labelled answers; give documents real authority values; smooth the single-claim case.
- **A second verifier** or numeric reasoning (units, magnitude, rounding) to raise recall on contradictions (0.44).
- **Pronoun retry:** re-check flagged pronoun sentences with the previous sentence attached.
- **Citations:** make the generator cite which chunk supports each sentence, then verify the citation.
- **Manual review of LLM attacks**, and running the attacks through the full pipeline.
- **Retrieval metrics** that suit 2-chunk documents (recall@k, hit rate); a chunk-size experiment; a larger corpus and more questions.
- **Attention check:** the mask-and-rerun test; a larger NLI model (more accurate, slower).
- **Done already (no longer future work):** threshold sweep, labelled-set precision/recall/F1 (AI-labelled), ablation of each component.

22. Question bank: likely questions and answers

Every answer follows the same pattern: **what we did, why, and its limit**.

22.1 The project as a whole

Q: What problem does your project solve?

AI models can state unsupported things confidently. HalluciGuard checks each sentence of a RAG answer against the sources, marks the unsupported parts, and gives a trust score.

Q: What is new compared to a normal RAG system?

Normal RAG stops after generating. We add sentence- and claim-level verification with NLI, a number and name checker that marks the exact wrong words, attention-based explanations, a hierarchical trust score, MI pruning, and an evaluation of the checker (adversarial test, labelled set and ablation).

Q: Do you detect false information?

No: we detect claims not supported by the sources. A true fact not in the documents is still unverified, and a wrong fact copied from a wrong document is marked supported.

Q: Did you train or fine-tune any model?

No. All models are used pretrained: DeBERTa NLI, MiniLM embeddings, spaCy and the qwen2.5:3b generator. The weights of the trust score and the thresholds are set by design.

22.2 Retrieval and pruning

Q: Why split documents into chunks?

Smaller pieces are easier to search precisely, and the checker can only read about 512 tokens at once.

Q: Why embeddings and not keyword search?

Embeddings match meaning, so different words with the same meaning are still found.

Q: Why is precision only 0.33?

Each document has about 2 chunks, so at most 2 of 6 retrieved chunks can be relevant: a ceiling of the setup. After pruning, precision is 0.74.

Q: What does PMI measure, and does pruning really help?

Whether the question's words are more common in a chunk than across all retrieved chunks. Yes: in the ablation, feeding NLI all 6 chunks lowered recall from 0.84 to 0.67 and F1 from 0.71 to 0.65.

Q: Why keep at least 2 chunks?

A safety net so pruning never leaves the generator without enough evidence.

22.3 Verification

Q: Why NLI and not an LLM judge?

It is local, fast, gives the same result every time, and outputs probabilities we can threshold. An LLM judge is slower and can hallucinate too.

Q: Why is the chunk the premise?

We are asking whether the source supports the answer, so the source is what we treat as known.

Q: Why 0.5? Did you tune it?

It is the midpoint where support becomes more likely than not, a design choice. We swept it from 0.3 to 0.9 on the labelled set: F1 stayed at 0.71-0.72, so the result is not sensitive to it; higher thresholds trade precision for recall.

Q: Why sentence by sentence, and what are claims?

Checking the whole answer hides a single wrong sentence. Compound sentences are further split into claims so one wrong fact is not hidden by a right one; the sentence takes its weakest claim's verdict.

Q: Your false-positive rate is 16.7%. Isn't that bad?

It is a main weakness (23% of supported sentences in the labelled set). Known causes: pronouns, long chunks, number formats. Smaller evidence snippets and a pronoun retry are the next fixes.

Q: Why a number checker? And did it help?

NLI is weak with exact numbers (a wrong figure once got 0.97 entailment). The checker confirms numbers and names exist in the source and marks the exact words. On our labelled set, which is mostly invented details and commentary, it did not raise overall F1, but it catches changed numbers and makes the explanation precise.

Q: Couldn't a number match in the wrong context?

Yes. We choose the passage the sentence is about by meaning, not by NLI, to reduce this, and NLI still judges the sentence; but a number that exists in the right passage attached to a different fact would pass.

Q: Why not fine-tune DeBERTa?

It needs a large labelled dataset and a GPU, and a strict split between training and test data; the proposal specified a pretrained model.

22.4 Attention and trust

Q: What does the attention view show?

The source words the checker focused on most for a sentence NLI rejected. It is an explanation aid, not proof of what caused the decision.

Q: Is this cross-attention?

No: DeBERTa uses self-attention over the joined chunk and sentence. We read the part from sentence tokens to chunk tokens.

Q: Why these trust weights?

Sentence support is the most direct evidence, so it has the largest weight. They are designed, not learned. Our analysis suggests token probability is stronger than its 0.20 weight implies and the document component is uninformative while all authorities are equal; learning the weights is future work.

Q: Why combine four signals?

Each has blind spots. NLI missed a wrong number that the entity check caught; the number check cannot judge meaning; model confidence alone does not show which sentence is wrong.

Q: What if a model gives no logprobs?

The token component's weight is dropped and the other weights are scaled to sum to 1.

22.5 Testing

Q: How do you know the checker works?

Three ways: (1) we corrupt true sentences and check detection (100% for negation, changed numbers and noise; 21% for weak LLM rewrites) with a control set for false positives; (2) a labelled set of 176 real answer sentences with precision, recall and F1; (3) an ablation of each component.

Q: Who labelled the data?

An AI assistant, reading each sentence against the source documents; it was not human-verified. We state that openly; a human re-label of a sample is the next step.

Q: Why compare against the clean chunk?

If we corrupted the documents, the generator would repeat the corruption and NLI would correctly call it supported, so nothing would be measured.

Q: Why a control set?

Without it, a checker that flags everything would look perfect.

Q: Why only 21% on LLM attacks?

The 3B model often wrote near-paraphrases rather than real contradictions, so the test is weak; subtle rewrites are also genuinely harder for NLI. This test also uses NLI alone.

Q: Isn't faulty or loose mode cheating?

No: a detector can only be shown on answers with errors. The modes are labelled, and normal mode stays strict.

Q: Why Ollama instead of Gemini?

The Gemini models we could use refused logprobs and had a small daily limit. Ollama has no limits, returns logprobs, and is private and reproducible. Gemini was removed from the project.

Q: How big is your evaluation?

22 questions, 6 documents and 176 labelled sentences: indicative, not a benchmark.

23. Wording guide: say this, not that

Say	Don't say	Why
“Detects claims not supported by the sources”	“Detects false information”	The system checks support, not truth
“Source words the checker attended to most”	“The words that caused the hallucination”	Attention isn't causal
“Self-attention over the joined pair”	“Cross-attention”	DeBERTa is a single encoder
“A probability for each label”	“Calibrated probabilities”	Calibration was never tested
“2 of 23 claim sentences”	Stressing “8.7%” alone	The sample is small
“F1 0.71 on a labelled set of 176 sentences, labelled by an AI assistant”	“Our system achieves 0.71 F1” without context	The labels are not human-verified; the intervals overlap with simpler variants
“Claim splitting and the number check didn't measurably change F1”	“Each component improved accuracy”	The ablation didn't show that
“The weights are designed, not learned”	Inventing a justification	Honesty is safer and respected
“100% on negation, stale facts and noise; 21% on weak LLM rewrites”	“Our system achieves 100% detection”	100% is only for simple attacks
“0.5 is a design choice; F1 is flat from 0.3 to 0.9”	“0.5 is the optimal threshold”	It is not tuned to an optimum
“Nothing is fine-tuned”	“We trained the model”	All models are pretrained

Part E

Running the demo and finding your way around the code

24. Running the demo

24.1 Before presenting

1. Start Ollama: `ollama serve` (and once: `ollama pull qwen2.5:3b`).
2. Start the backend: `.venv/bin/python -m uvicorn app.main:app --port 8000`. It preloads the models at startup (about 12 seconds).
3. Start the dashboard: `.venv/bin/python -m streamlit run dashboard/streamlit_app.py` and open http://localhost:8501.
4. Using the `.venv/bin/python -m ...` form avoids starting a global Python that lacks the packages.
5. Run one test question so everything is warm.
6. Test your questions the day before, and keep screenshots or a screen recording as a backup.
7. For a loose-mode demo, start the backend with `PROMPT_MODE=loose` and restart it without afterwards.

24.2 Demo script (about 3 minutes)

1. Ask a normal question. Show the green sentences, the grey filler sentence, the entity list and the trust score. Point out “6 retrieved → 2 used”.
2. Tick **Faulty mode** and ask again. Show the bright red wrong numbers inside the amber sentence, the struck-through original, the “Caught” line, the lower trust score, and the source passage in the expander.
3. Open the **Robustness** tab: detection per attack type and the false-positive rate.
4. Open the **Evaluation** tab for the headline numbers (this calls the generator for all 22 questions, so run it beforehand or rely on `README.md`).
5. Mention the ablation table (section 18.3) and be ready to say the labels were made by an AI assistant.

> **Tip**
> Good demo questions: one the documents answer fully (all green); one they answer only partly; and one completely off-topic (“Who won the 2022 football World Cup?”), where the model should say the context has no answer and the sentence appears as grey filler.

> **Be careful**
> In faulty mode a one-claim answer can show a trust score of 0.00. That is the formula working as designed (section 13.9), but be ready to explain it.

24.3 Troubleshooting

Problem	Likely cause	Fix
Dashboard shows a connection error	Backend not running	Start uvicorn
ModuleNotFoundError (e.g. nltk) when starting	Started with a Python outside the project's `.venv`	Use `.venv/bin/python -m uvicorn ...`

Query fails with a connection error to Ollama	Ollama server not running	`ollama serve`
First query very slow	Models not preloaded (`WARMUP=0`) or Ollama unloaded the model	Run a test query; keep-alive is 30 minutes
“database is locked”	Another process holds `app.db`	Wait (SQLite waits up to 30 s) or close the other process
Everything slow	Memory full (swapping)	Close other programs; restart
Answer all green in faulty mode	Checkbox not ticked	Tick it and ask again

25. Where things are in the code

For an exact, line-by-line list of every threshold, rule and number, see **FACTS.md** in the project root.

File	What it does
app/main.py	Starts the FastAPI backend, connects the routes, preloads models
app/config.py	Settings: model names, thresholds, weights, prompts (a few constants still live in other files; see FACTS.md)
app/db.py, app/models.py	The SQLite database and its tables (and the small migration that adds new columns)
app/routers/	API endpoints: query, ingest, evaluate, adversarial
app/core/chunking.py, embeddings.py, vector_store.py, ingestion.py	Cut documents into chunks, embed them, store and search them
app/core/mi_pruning.py	PMI scoring and pruning
app/core/rag_pipeline.py	Retrieve → prune → generate (with the cache and logprobs)
app/core/query_service.py	The full /query flow: runs the pipeline, verification, attention, trust, and saves everything; also rescoring
app/core/verification.py	The sentence check used by the app and the ablation: NLI, claims, filler, wrong parts
app/core/nli_verifier.py	Loads DeBERTa once (`get_nli_model`) and runs NLI
app/core/sentence_localizer.py	Sentence splitting, claim-level NLI verdicts, filler detection
app/core/claim_extractor.py	Splits compound sentences into claims
app/core/span_checker.py	Finds numbers and names the sources do not contain
app/core/attention_extractor.py	Top attention words for NLI-rejected sentences
app/core/trust_engine.py	Entity check and the four trust components
app/core/adversarial_agent.py	Attacks, the attack suite, and faulty-mode corruption
app/core/evaluation.py, metrics.py	Evaluation metrics; retrieval-only and rescore modes
app/core/warmup.py	Preloads models at startup
dashboard/streamlit_app.py	The dashboard
scripts/seed_corpus.py	Loads the documents (run once)
scripts/run_eval.py, run_adversarial_eval.py	Evaluation and the adversarial suite from the command line
scripts/build_labeled_set.py, run_ablation.py	Generate candidate answers; run the labelled-set evaluation and ablation (about 25 minutes the first time)
scripts/compare_generators.py	Benchmarks the generator (hallucination rate, trust, latency)
data/raw_docs/, data/eval/	The 6 documents; the 22 questions; the labelled sentences

outputs/	Saved results: ablation.json, generator_comparison.json
tests/	43 automated tests (pytest, on a temporary database)
FACTS.md, PROGRESS.md, README.md	Exact rules and numbers; build log and assumptions; setup and results

25.1 Suggested reading order for the code

1. **query_service.py** — the whole flow in one place.
2. **verification.py** — how a sentence gets its colour.
3. **trust_engine.py** — how the score is built.
4. **FACTS.md** — the exact rules; **PROGRESS.md** — the assumptions made during the build (professors often ask about these).

26. Glossary

Term	Meaning
Ablation	Switching one component off to measure what it contributes
Adversarial test	Deliberately corrupting sentences to test whether the checker notices
AI-labelled	Labels made by an AI assistant instead of a human
API / endpoint	An address a program sends requests to (e.g. POST /query)
Attention	How strongly a transformer connects each word to the other words
Authority score	A 0-1 trust value for each source document (all 0.8 here)
AUROC	Probability that a bad example scores worse than a good one; 0.5 = chance
Bootstrap confidence interval	A range for a number from many random re-samplings of the data
Cache	Saved answers reused for identical prompts
ChromaDB	A database that stores embeddings and finds the nearest ones
Chunk	A ~250-word piece of a document
Claim (atomic)	One checkable fact inside a sentence
Claim sentence	A sentence that is not filler
Control set	Unchanged true sentences used to measure false positives
Corpus	The collection of source documents (6 here)
Cosine similarity	How similar two embeddings are
DeBERTa	The transformer that performs NLI
Detection rate	Fraction of corrupted sentences caught
Embedding	A list of numbers representing a text's meaning
Entailment / neutral / contradiction	The premise supports / doesn't address / opposes the hypothesis
Entity	A specific name, number or date in a sentence
F1	The balance of precision and recall
Faithfulness / groundedness	Whether a claim is supported by the sources
False positive	A true sentence wrongly flagged
Faulty mode	Test mode that corrupts an answer on purpose
Filler sentence	A meta sentence with no claim (“this is stated in the context”)
Generator	The AI model that writes the answer (qwen2.5:3b)
Hallucination	In this project: a claim not supported by the sources

Hypothesis / premise	In NLI: the claim being checked / the source text
Keep-alive	Ollama's setting for how long a model stays in memory (30 minutes)
Labelled set	176 answer sentences marked supported or unsupported, used to measure P/R/F1
Localisation	Finding exactly which sentence (or word) is unsupported
Logprob	The logarithm of the probability the model gave a word
Loose mode	Test mode with a prompt that invites outside knowledge
MiniLM	The small model that creates embeddings
NLI	Natural language inference: does text A support text B?
Ollama	A program that runs AI models locally
PMI	Pointwise mutual information: is a word more common in this chunk than on average?
Precision / recall	Of the items flagged, the fraction that were right / of the true items, the fraction found
Pruning	Removing chunks that aren't useful
RAG	Retrieval-Augmented Generation: look up relevant text, then answer from it
Reference chunk	The chunk a sentence is about, chosen by meaning; used by the number check
Renormalisation	Rescaling weights to add up to 1 when one is missing
Softmax	Turns raw model scores into probabilities
Span check	The number and name check that marks wrong words
spaCy	A library that finds entities and grammar in text
SQLite	A simple database stored in one file
Streamlit	The tool used to build the dashboard
Temperature	How random the generator is; 0 = always the most likely word
Threshold	The cut-off (0.5) below which a sentence is flagged
Token	A word or piece of a word, the unit models read
Transformer	The neural-network type behind modern language models
Trust score	The 0-1 summary of an answer's reliability
Warm-up	Loading all models at startup so the first query is fast

Appendix: the running example, end to end

Step	What happened to “How much new solar capacity was added globally in 2023?”
Retrieval	6 chunks closest in meaning were found, including some about wind and policy
MI pruning	Chunks that don't concentrate the question's words were dropped: 6 → 2 kept (a solar chunk and a policy chunk)
Generation	qwen2.5:3b wrote: *“In 2023, about 345 gigawatts of new solar capacity was added globally. This information is directly stated in the context provided.”* (about 2.5 s)
Filler detection	The second sentence has no number and mentions “context” and “stated”: filler, grey, ignored in scoring
Claims + NLI	The first sentence is one claim; its best chunk is the solar chunk, entailment well above 0.5: green
Number/name check	345 and 2023 are in the solar chunk: nothing marked
Attention	Not needed: no sentence rejected by NLI
Trust	Sentence about 0.9, entity 1.0, document 0.8, token probability about 0.9: trust about 0.9
Faulty mode (same question)	The answer becomes “In 2013, about 690 gigawatts...”; NLI still says entailment (0.80), but 2013 and 690 are not in the solar chunk: both marked bright red, sentence amber, trust 0.00 (one claim)

Final self-check

Can you explain, in your own words: (1) why the answer is checked sentence by sentence and claim by claim; (2) why a true sentence can be red; (3) why the adversarial test uses the clean chunk and what it does not cover; (4) why model confidence is only 20% of the trust score, and what the ablation says about that; (5) who labelled the evaluation data? If yes, you understand the project.